Learning Agent Code Report

Time-Aware Reinforcement Learning for Ball Catching under Visual Delay

ME5418 Group 30

1 Code Explanation

The learning agent connects the two previous milestones: it plays a network from `networks/` in the Gym environment of `ball_catch/` and trains it with PPO. The new code is a package `agent/`, a training script `train.py`, a parallel rollout collector added to `networks/ppo.py`, an input-standardisation layer in the networks and a `--model` option for `demo.py`: about 600 lines of code and 21 new unit tests (162 in the repository).

1.1 Organisation

File	What it does
`agent/ppo_agent.py`	`TrainConfig` (every hyperparameter, saved with the model) and `PPOAgent`: `update`, `train`, `evaluate`, `save`; `load_policy`.
`agent/controller.py`	`NetworkController`: a trained network behind the interface of the hand-written controllers, so that it is evaluated and displayed by the same code.
`networks/ppo.py`	New `RolloutCollector` (several environments, one batched network call per step) and `Rollout.select` (a minibatch of environments).
`train.py`	Command line: training log (CSV), learning curve, checkpoint, final comparison.
`demo.py --model`	Watches or evaluates a trained network in the live window, a GIF, or statistics only.

1.2 One PPO update

1. Collect. 16 environments run in lockstep for 128 control steps (2,048 steps, about 38 throws). At each step one network call returns, for all environments, the sampled actions, their log-probabilities, the values and the new GRU memory. A finished throw is reset at once and marked `episode_start`, where the network clears its memory. Throws and memory carry over to the next rollout.

2. Advantages. GAE with γ and λ from the table below; there is no bootstrapping across the end of a throw, and the critic's value after the last step bootstraps the unfinished ones.

3. Learn. 10 epochs over the rollout, each split into 4 minibatches of 4 environments. A minibatch keeps every sequence whole, and the GRU is re-run from the memory it had at the start of the rollout, so before the first gradient step the re-computed log-probabilities equal the stored ones (a unit test checks this). Each minibatch takes one Adam step on the clipped loss of the previous milestone, with the gradient norm clipped to 0.5 and the learning rate decayed linearly to zero.

4. Inputs. A new first layer, `ObservationNorm`, standardises each input with the running mean and variance of all observations so far. It is updated only after the learning phase, so a rollout is collected and re-evaluated with the same statistics, and it is saved with the weights.

Rollout	16 environments × 128 steps	Discount γ, GAE λ	0.995, 0.95
Learning	10 epochs × 4 minibatches of 512 steps	Clip, value / entropy weight	0.2, 0.5 / 0
Adam	lr 3×10^{-4} decayed linearly to 0, $\epsilon = 10^{-5}$	Gradient norm	clipped to 0.5
Budget	3 M steps per network (1,465 updates)	Inputs	running standardisation

1.3 Evaluation and first results

`PPOAgent.evaluate` plays the mean action (no exploration noise) on held-out throws seeded from 100,000, while training environments start from seeds 0–15. Throws, sensor noise and delays depend only on the seed, so the controllers of the first milestone face exactly the same episodes. Each network was trained once (seed 0) for 3 M steps under jittered delay, about 30 minutes per recurrent network on one CPU core. All three now catch more balls than the hand-written chase controller, and the two recurrent networks clearly more than the single-frame one (Fig. 1, Table 1). The single-frame network stopped improving after about 1.5 M steps. The temporal network learned more slowly than the recurrent one during the first 1.3 M steps, then overtook it. With one training run per network, the 4.9-point gap between them is not yet evidence that timestamps help. The miss distance (0.36 m against a catch radius of 0.23 m) shows that the networks move towards the ball but do not yet predict the crossing point accurately; the delayed oracle still catches 97%.

How to run.

```
python train.py                          # temporal network, jitter delay -> runs/temporal_jitter_seed0/
python train.py --variant recurrent      # or single_frame; --help lists every option
python demo.py --model runs/temporal_jitter_seed0/model.pt   # watch the trained network
python -m unittest                       # all 162 tests
```

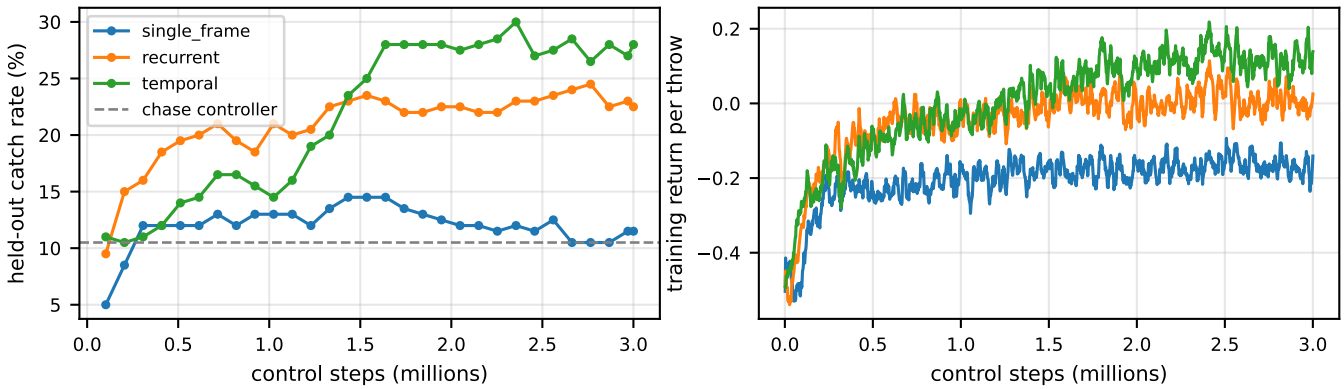


Figure 1: Training of the three networks (jitter delay, seed 0). Left: catch rate of the mean action on 200 held-out throws every 50 updates. Right: return of the training throws (exploring policy), averaged over 10 updates. Made with `report/plot_learning_curves.py` from the logs written by `train.py`.

Table 1: The same 1,000 held-out throws (jitter delay; networks use their mean action).

Controller	Uses	Caught	Mean return	Mean miss distance
Stationary	nothing	5.3%	−0.41	0.70 m
Chase (hand-written)	latest frame	10.5%	−0.27	0.55 m
`single_frame` network	latest frame, net state	14.8%	−0.15	0.49 m
`recurrent` network	+ memory	23.7%	+0.04	0.41 m
`temporal` network	+ memory, capture interval, age	28.6%	+0.13	0.36 m
Oracle (privileged, delayed start)	true crossing point	97.4%	+1.37	0.03 m

2 Existing Codes and Libraries

- **PyTorch 2.5.1**: Adam, `clip_grad_norm_`, saving and loading of weights; **Matplotlib** for the learning curves. No new dependency.
- **Our environment and networks** (previous milestones), used through `BallCatchEnv`, `make_policy`, `act` and `evaluate`, and `ball_catch.evaluation`.
- **Method references**: PPO (Schulman et al., 2017) and GAE (Schulman et al., 2016). The update follows the CleanRL single-file implementations: Adam with $\epsilon = 10^{-5}$, gradient clipping, linear learning-rate decay and the KL estimate $\mathbb{E}[(r-1) - \log r]$ from `ppo_continuous_action.py`; minibatches of whole environment sequences with their initial memory from `ppo_atari_lstm.py`. We wrote our own code from these descriptions.

3 Design Choices and Next Steps

Design choices.

- **Minibatches of whole sequences.** A throw lasts at most 72 steps and a rollout 128, so each minibatch simply re-runs complete 128-step sequences from the stored memory. This avoids truncated back-propagation and burn-in, and keeps re-evaluation exact.
- **Throws continue across rollouts.** Restarting every environment at each rollout would over-represent the beginning of throws, and the last throw of every rollout would be cut short.
- **One CPU thread.** For networks this small, PyTorch's multi-threading made updates 25–30% slower; one thread per run lets the three networks train in parallel.

Tuning (temporal network, seed 0). With 4 epochs and lr $= 3 \times 10^{-4}$ the KL divergence per update was about 10^{-3} and only 6% were caught after 1 M steps. Ten epochs reached 31% after 2 M steps, lr 10^{-3} with four epochs 21%. With 10^{-3} and ten epochs the catch rate peaked at 25% and then fell to 14% as the action noise collapsed (σ: $0.6 \to 0.14$), and the single-frame network drove the net into a corner (2%). $\lambda = 0.99$ was not faster than 0.95. Input standardisation improved the held-out return early on (−0.22 against −0.57 after 0.1 M steps); the runs were level by 0.8 M steps.

Next steps. Several seeds per network before comparing them; evaluation across fixed delays of 0–0.2 s and on unseen burst delays; the predictive estimator with a PD controller as physics-based baseline; and a check of action saturation (mean actions beyond ±1), where tanh squashing is the fallback.

AI tool declaration. We used Claude and GPT, through the AI assistant, to help design, write and test the agent code, and to draft and optimise this report. The requirements for the project were proposed by us after discussion. We are responsible for the content and quality of the submitted work.